Introduction to Machine Learning (67577)

Exercise 4
Regularization & CNN

Second Semester, 2026

Contents

Submission

Please make sure to follow the general submission instructions available on the course website. In addition, for the following assignment, submit a single `ex4_ID.zip` file containing:

- An `Answers.pdf` file with the answers for all theoretical and practical questions (include plotted graphs *in* the PDF file).
- The following python files (without any directories):
  `gradient_descent.py`
  `learning_rate.py`
  `logistic_regression.py`
  `modules.py`
  `regularization_main.py`
  `train_cifar_cnn.py`

The `ex4_ID.zip` file must be submitted in the designated Moodle activity prior to the date specified *in the activity*.

- Plots included as separate files will be considered as not provided.

1 Theoretical Part

1.1 Regularization

1. This question compares Ridge Regression with the standard Least Squares (LS) estimator—the estimator obtained from the familiar closed-form solution of linear regression. We will show that even though the Ridge estimator is biased, it can achieve lower generalization error compared to the LS estimator, in the presence of noisy data.
   We discussed in class the concepts of bias and variance in an abstract sense. In some cases, for a given estimator $\hat{\theta}$ and a target parameter θ, we can define and compute the bias and variance using mathematical formulas as follows:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$$

$$\text{Var}(\hat{\theta}) = \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])(\hat{\theta} - \mathbb{E}[\hat{\theta}])^\top\right]$$

   An estimator $\hat{\theta}$ is said to be unbiased if $\text{Bias}(\hat{\theta}) = 0$.
   Let $\mathbf{X} \in \mathbb{R}^{m \times d}$ be a **constant** design matrix, $\mathbf{y} \in \mathbb{R}^m$ a response vector, and assume that $\mathbf{X}^\top\mathbf{X}$ is invertible. Denote $\hat{\mathbf{w}}$ the LS solution and $\hat{\mathbf{w}}_\lambda$ the ridge solution for the hyperparameter $\lambda \geq 0$.
   - Assume the linear model is correct, namely $\mathbf{y} = X\mathbf{w} + \varepsilon$ where $\varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$.
   - Note that in this case, LS estimator is unbiased:

$$\begin{aligned}\mathbb{E}[\hat{\mathbf{w}}] &= \mathbb{E}[(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}] = \mathbb{E}[(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top(\mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon})] \\ &= \mathbb{E}[\mathbf{w} + (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\boldsymbol{\varepsilon}] = \mathbf{w} + (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{w}\end{aligned}$$

   (a) Show that $\hat{\mathbf{w}}_\lambda = A_\lambda\hat{\mathbf{w}}$ where $A_\lambda := (\mathbf{X}^\top\mathbf{X} + \lambda I_d)^{-1}(\mathbf{X}^\top\mathbf{X})$
   (b) From the above, conclude that for any $\lambda > 0$, the ridge estimator is a biased estimator of $\mathbf{w}$. That is, show that for any $\lambda > 0$ we have $\mathbb{E}[\hat{\mathbf{w}}_\lambda] \neq \mathbf{w}$.

(c) Show that: $\text{Var}(\hat{\mathbf{w}}_\lambda) = \sigma^2 A_\lambda (\mathbf{X}^\top \mathbf{X})^{-1} A_\lambda^\top$, for σ^2 the variance of the assumed noise.
Hint: Note that for a constant matrix B and a random vector $\mathbf{z}$ it holds that $\text{Var}(B\mathbf{z}) = B \cdot \text{Var}(\mathbf{z}) \cdot B^\top$ and that $\text{Var}(\hat{\mathbf{w}}) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$.

(d) In the multivariate case, the Mean Squared Error (MSE) of $\hat{\mathbf{w}}_\lambda$ is defined as:

$$\mathbb{E}\left[\|\hat{\mathbf{w}}_\lambda - \mathbf{w}\|^2\right] = \mathbb{E}\left[(\hat{\mathbf{w}}_\lambda - \mathbf{w})^\top (\hat{\mathbf{w}}_\lambda - \mathbf{w})\right]$$

where $\mathbf{w}$ is the true parameter vector, and $\hat{\mathbf{w}}_\lambda$ is our estimator.
Derive an explicit expression for the MSE of $\hat{\mathbf{w}}_\lambda$ as a function of λ.

(e) The derivative of the MSE from the previous subquestion with respect to λ is negative at $\lambda = 0$ assuming the data is noisy (we omit here the proof). Conclude that, if the linear model is correct, a little Ridge regularization helps to reduce this generalization error.

2. Recall that in the recitation, we examined Tikhonov regularization for linear regression:

$$\hat{\mathbf{w}}_{\mathbf{L}}^{Tik} = \arg\min_{\mathbf{w}\in\mathbb{R}^d} \left(\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \|\mathbf{L}\mathbf{w}\|^2\right)$$

with some matrix $\mathbf{L}$. In this exercise, we will further explore this regularization and its properties, particularly in comparison with ridge regularization.

(a) Let $\mathbf{X} \in \mathbb{R}^{m\times d}$, $\mathbf{y} \in \mathbb{R}^m$, and $\mathbf{L} \in \mathbb{R}^{k\times d}$. Assume that $\mathbf{X}^\top \mathbf{X}$ is invertible. Find a closed-form solution for the LS problem with Tikhonov regularization for $\mathbf{X}, \mathbf{y}, \mathbf{L}$ as given above. That is, find $\hat{w}_{\mathbf{L}}^{Tik}$ as defined above.

(b) Is the condition that $\mathbf{X}^\top \mathbf{X}$ is invertible a necessary requirement to find the solution derived in question (a)? If not, is there a weaker condition that can be assumed to obtain the same solution?

(c) **Solving Least Squares with and without regularization** - Consider the following data matrix $\mathbf{X}$ and the vector of labels $\mathbf{y}$:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

solve the least squares (LS) problem on this data in three different ways:

i. Without regularization
ii. With ridge regularization, using $\lambda = 1$
iii. With Tikhonov regularization, using the given matrix: $\mathbf{L} = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$

Hint - use the fact that: $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

Then, analyze the results:

i. Compare the norms of the three weight vectors: which one is the largest, which one is the smallest, and why?
ii. Compare the absolute values of the second coordinate of the three weight vectors: which one is the largest, which one is the smallest, and why?

1.2 CNN

1. Consider a convolutional neural network (CNN) applied to RGB images of size $32 \times 32 \times 3$ (i.e., 3 input channels):

- **Conv1:** 16 filters of size 5×5, stride 1, padding 2
- **ReLU**
- **MaxPool1:** 2×2, stride 2
- **Conv2:** 32 filters of size 3×3, stride 1, padding 1
- **ReLU**
- **MaxPool2:** 2×2, stride 2
- **Flatten**
- **Dense1:** Fully connected layer with 128 units
- **Dense2:** Fully connected layer with 10 units (for classification)

(a) For each of the following layers, compute the number of trainable parameters (assuming there are no bias terms):
- Conv1
- Conv2
- Dense1
- Dense2

(b) Compute the shape (height $\times$ width $\times$ channels) of the output after each layer (Conv, Pool, Dense).

(c) What is the total number of trainable parameters in the network?

2. Consider the following 3×3 convolutional filter:

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

(a) What kind of image pattern or feature does this filter respond to?

(b) Give an example of a 3×3 input patch where this filter produces a large positive value.

(c) What kind of patch would make the output close to zero?

3. Consider the following CNN architecture applied to a 64×64 input image (single channel). The architecture is as follows:
- **Layer 1:** 5×5 convolution, stride 1
- **Layer 2:** 2×2 max pooling, stride 2
- **Layer 3:** 3×3 convolution, stride 1
- **Layer 4:** 2×2 max pooling, stride 2

Compute the receptive field size of a single output cell in Layer 4 with respect to the input image.

2 Practical Part

2.1 Minimizing Regularized Logistic Regression

In the following part you will use Gradient Descent to solve a regularized logistic regression optimization problem. We will see how a convex optimization problem can be solved using gradient based methods.

Note: You will need to use some of your implementations from ex3. In the `modules.py` file, copy your implementations for the `L1` and `L2` classes (note that the file contains some other modules in this exercise, so make sure you do not delete them!). In addition, copy your implementation for the `gradient_descent.py` and `learning_rate.py` files. If you haven't completed ex3, please refer

to the instructions in ex3 and complete those files now.
After copying you code from ex3, implement the following:

- Implement the `LogisticModule` in the `modules.py` file, as described in class documentation.
  - In the `compute_output` function, you should return the negative log-likelihood

$$f(\mathbf{w}) = -\frac{1}{m}\log\left(\prod_i P(Y = y_i \mid X = \mathbf{x}_i, \mathbf{w})\right)$$

    recall derivations seen in class and recitations, as well as in the course book.
  - In the `compute_jacobian` function, you should return the derivative of the objective above $\frac{\partial f(\mathbf{w})}{\partial \mathbf{w}}$.
- Implement the `RegularizedModule` in the `modules.py` file, as described in class documentation. This module receives two generic modules to be used as the fidelity term (i.e. the loss function) and regularization term. For example: `RegularizedModule(LogisticModule(), L1())`.
- Implement the `LogisticRegression` class in the `logistic_regression.py` file as specified in class documentation. This class should wrap the usage of your gradient descent implementation on the `LogisticModule`, `RegularizedModule`, `L1` and `L2`.
- When initializing the model's weights sample from $\mathbf{w} \sim \mathcal{N}\left(0, \frac{1}{d}I\right)$. This is equivalent to sampling from the standard Gaussian distribution and dividing by $\sqrt{d}$.

R Notice that the `LogisticRegression` class does not create an instance of the `GradientDescent` class. Instead, it receives it as a *dependency* in the constructor. As such, your `LogisticRegression` implementation is open for future extensions and support of different solvers. This is one of the 5 SOLID coding principles also known as the open-close principle.

Then, in the `regularization_main.py` file load the South Africa Heart Disease dataset (SA-heart.data), split it to train- and test sets (80% train) and answer the following questions:

4. Using your implementation, fit a logistic regression model over the data. Use the `predict_proba` to plot an ROC curve. You can use sklearn's `metrics.roc_curve` function and the code provided in Lab 04.
5. Which value of α achieves the optimal ROC value according to the criterion below. Using this value of α^* what is the model's test error?

$$\alpha^* = \text{argmax}_\alpha\{\text{TPR}_\alpha - \text{FPR}_\alpha\}$$

6. Fit an ℓ_1-regularized logistic regression by passing `penalty="l1"` when instantiating a logistic regression estimator, with $\alpha = 0.5$. For each value $\lambda \in \{0, 0.001, 0.002, 0.005, 0.01, 0.02, 0.05\}$, fit the model and save the train- and test- error. Plot the train and test errors as a function of λ. You can (but don't have to) use `max_iter=20,000`, `lr=1e-4` and the default values for `out_type` and `tol`.
7. Repeat question 6 for ℓ_2-regularized logistic regression and plot the results respectively. Which value of λ performed best?

2.2 Designing a CNN for CIFAR-10

Your task is to design and train a **convolutional neural network (CNN)** that achieves at least **70% accuracy** on the CIFAR-10 dataset.

> R The **CIFAR-10** dataset is a classic benchmark in computer vision, created by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton at the University of Toronto. It was introduced in 2009 as a simplified version of the larger Tiny Images dataset to support fast experimentation with image classification models. CIFAR-10 consists of 60,000 color images of size 32×32 pixels, categorized into 10 object classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Despite its relatively small image size and simple structure, CIFAR-10 remains a widely used benchmark for testing and comparing deep learning models. A more challenging variant, CIFAR-100, is built on the same image set but includes 100 finer-grained classes, making it a more difficult benchmark for modern models. While this assignment focuses on CIFAR-10, CIFAR-100 can be a good next step for experimenting with more complex architectures or fine-grained classification.

We provide a skeleton file, `train_cifar_cnn.py`, which includes a data loading function and minimal documentation. Unlike most previous practical exercises, you are not required to implement the network components from scratch: you are encouraged to use existing modules such as `Conv2d` from `torch.nn`.

Additionally, the provided skeleton is intentionally minimal and does not enforce a specific API. This is designed to simulate a real-world scenario, giving you the freedom to structure your code and training pipeline as you see fit.

You are encouraged to experiment with the following aspects of your model:

- **CNN architecture** – Try varying the number of layers, filter sizes, activation functions, and the use of regularization techniques such as dropout.
- **Training set size** – You may choose to train on a subset of the training data, but make sure to evaluate your model on at least **1,000 test examples**.
- **Data augmentation** – Techniques such as random cropping, horizontal flipping, and color jittering are widely used in practice to improve generalization in image classification tasks. You are welcome to explore these on your own if you're interested. Note that data augmentation is entirely optional — you can achieve the required 70% accuracy without it.

Your complete training pipeline should run in **under 10 minutes** on your machine. If needed, you may reduce the training set size or simplify your model accordingly.

What to Include in Your Solution PDF:

Your submitted report must include:

1. **Final Model Description**
   Clearly describe your final architecture and training configuration, including the optimizer, loss function, and final test accuracy.
2. **Experimental Process**
   Summarize what you tried during the development process: what worked, what didn't, and how you improved your model.
3. **Runtime and Environment**
   Report the total training time (in minutes) and describe the hardware used (e.g., CPU/GPU

model, RAM).

If you do not reach 70% accuracy despite multiple reasonable attempts, describe your efforts and what you learned. You are welcome to discuss ideas with classmates (but **do not share or view each other's code**) and consult online sources for general guidance.

Implementation Notes — Applicable Only to This Question

For this question only (Question 2.2), you are allowed to use **PyTorch** and standard modules from `torch.nn`, including components such as `Conv2d`, `ReLU`, `MaxPool2d`, `Dropout`, `Linear`, and others.

You may also import any standard Python or PyTorch libraries needed for training, evaluation, and visualization.